

Bloc 2 — Data Architecture Design

RetailFlow Data Architecture Plan

Executive Summary

I designed the RetailFlow Platform as an end-to-end Retail Intelligence architecture for modern e-commerce organizations.

The purpose of this data architecture is to transform customer events, operational retail data and machine learning outputs into a governed, observable and decision-ready platform.

The architecture combines:

- a modular Docker-based infrastructure;
- a PostgreSQL analytical and operational data layer;
- a Redpanda Kafka-compatible streaming broker;
- a FastAPI service layer;
- a Streamlit business and monitoring interface;
- Apache Airflow orchestration;
- Prometheus and Grafana observability;
- PostgreSQL exporter metrics;
- GitHub Actions CI/CD workflows;
- a future-ready cloud and Kubernetes deployment path.

The architecture is designed to support the following core capabilities:

Capability	Architectural Response
Real-time customer event processing	Redpanda, FastAPI event producer, Python event consumer
Reliable storage	PostgreSQL with separated schemas
Data governance	Dedicated governance schema, retention logs, consent data, quality logs
Customer intelligence	Analytics schema, ML predictions, customer segments
ML serving	FastAPI endpoints exposing model outputs and reports
Orchestration	Airflow DAGs for quality, sales aggregation, ML retraining and retention cleanup
Observability	Prometheus, Grafana, PostgreSQL exporter, application metrics
Developer workflow	Git, GitHub, Docker Compose, GitHub Actions CI/CD

This document presents the architecture from both a conceptual and implementation point of view.

It explains the main design decisions, the infrastructure components, the data model, the deployment approach, the monitoring layer and the future architecture roadmap.

1. Architecture Objectives

I designed the RetailFlow architecture around the idea that a data platform should not only store data.

It should provide a complete operating environment where events become trusted data, trusted data becomes intelligence, and intelligence becomes actionable business insight.

The main objectives are:

1. provide a coherent end-to-end data platform;
 2. support real-time event ingestion;
 3. separate operational, raw, analytical and governance data layers;
 4. make customer intelligence accessible through APIs and dashboards;
 5. support orchestration and automation;
 6. provide platform observability;
 7. support governance, privacy and auditability by design;
 8. remain reproducible through Docker Compose;
 9. support a future migration toward cloud and Kubernetes.
-

2. Business and Technical Context

RetailFlow is a Retail Intelligence platform designed for e-commerce organizations.

The platform captures and processes customer activity such as:

- product views;
- add-to-cart events;
- checkout events;
- purchases;
- returns;
- support interactions;
- reviews;
- customer consent updates;
- browsing behavior.

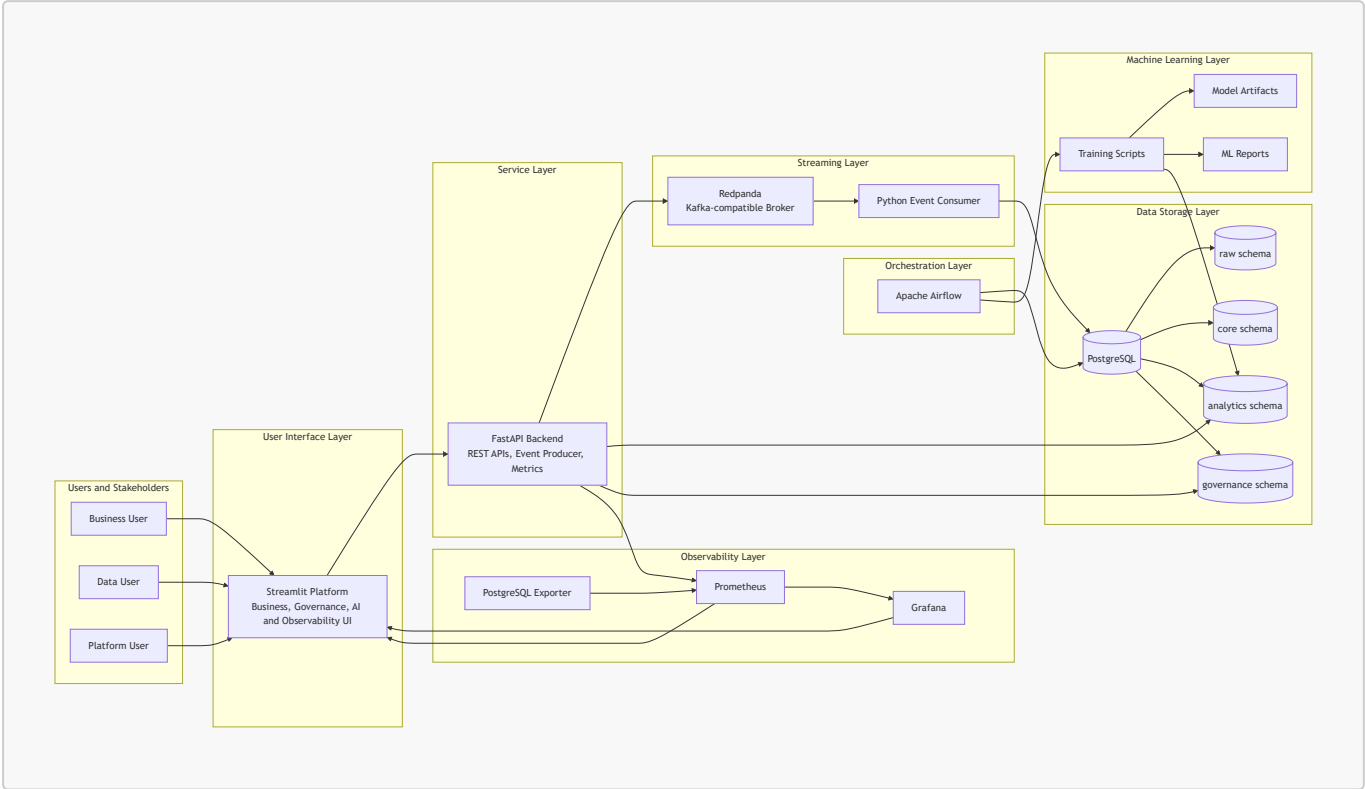
The platform must support business questions such as:

- Which customers are most valuable?
- Which customers are at risk of churn?
- Which customer segments require different actions?
- Are live customer events being ingested correctly?
- Are invalid events isolated and traceable?
- Are ML models monitored and explainable?
- Are platform components healthy and observable?

To answer these questions, I designed a modular architecture where each component has a clear responsibility.

3. High-Level Architecture

The RetailFlow architecture follows a layered approach.



This architecture separates responsibilities across several layers.

Layer	Role
User interface	Exposes the platform to users through Streamlit
API layer	Provides service access to data, events, ML and governance
Streaming layer	Decouples event production and event persistence
Storage layer	Centralizes structured data in PostgreSQL
Orchestration layer	Automates scheduled workflows
Machine learning layer	Trains, stores, reports and serves customer models
Observability layer	Monitors services, metrics and platform health

4. Architecture Design Principles

4.1 Modularity

I separated the platform into specialized services.

Each component has a clear role:

Component	Responsibility
Streamlit	User interface and dashboard layer
FastAPI	Backend API and event publishing layer
Redpanda	Event broker
Event consumer	Validation and persistence of events
PostgreSQL	Central storage and analytical database
Airflow	Workflow orchestration
ML scripts	Training, prediction and monitoring artifacts
Prometheus	Metrics collection
Grafana	Metrics visualization
GitHub Actions	CI/CD automation

This modularity makes the platform easier to maintain and easier to explain.

4.2 Separation of Concerns

I designed the architecture so that each layer solves a different problem.

For example:

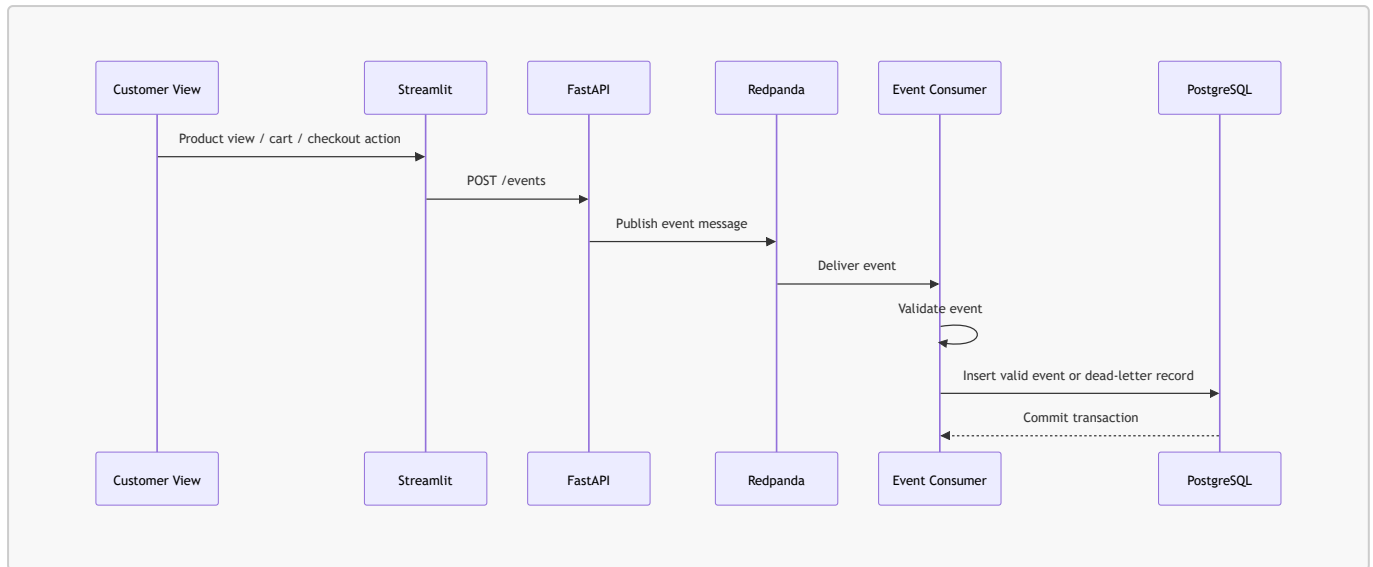
- Streamlit does not write directly to the database for live events.
- FastAPI produces events but does not persist them directly into the raw event table.
- Redpanda decouples event publishing from consumption.
- The consumer validates and persists events.
- PostgreSQL stores trusted, analytical and governance data.
- Airflow runs scheduled jobs.
- Prometheus and Grafana monitor the platform.

This separation supports scalability, maintainability and reliability.

4.3 Event-Driven Architecture

The customer journey is event-driven.

A customer action generates an event that flows through the platform.



This design is closer to production e-commerce platforms than direct synchronous database writes.

It provides:

- decoupling;
- resilience;
- extensibility;
- traceability;
- clear data quality control points.

4.4 Governance by Design

I integrated governance directly into the architecture.

Governance is represented through:

- consent fields;
- retention policies;
- retention action logs;
- anonymization workflow;
- data quality logs;
- dead-letter events;
- API governance endpoints;
- Streamlit governance dashboards.

This makes governance operational rather than purely documentary.

4.5 Observability by Design

RetailFlow exposes platform metrics and health information.

Observability is available through:

- FastAPI `/metrics` endpoint;
- Prometheus scrape configuration;

- PostgreSQL exporter;
- Grafana dashboards;
- Airflow health endpoint;
- Streamlit Observability page.

This provides visibility into platform state and operational reliability.

4.6 Local Reproducibility

The platform runs locally through Docker Compose.

This enables:

- consistent setup;
- reproducible deployment;
- simplified evaluation;
- easier debugging;
- clear service orchestration.

4.7 Future-Ready Deployment

Although the current environment is Docker Compose-based, I designed the architecture so that it can evolve toward:

- Kubernetes;
- cloud-managed PostgreSQL;
- managed Kafka-compatible streaming;
- cloud monitoring;
- container registry deployment;
- production-grade IAM and SSO.

The `/k8s` directory is considered part of a future deployment roadmap rather than the current primary runtime.

5. Runtime Infrastructure

RetailFlow runs as a multi-container platform.

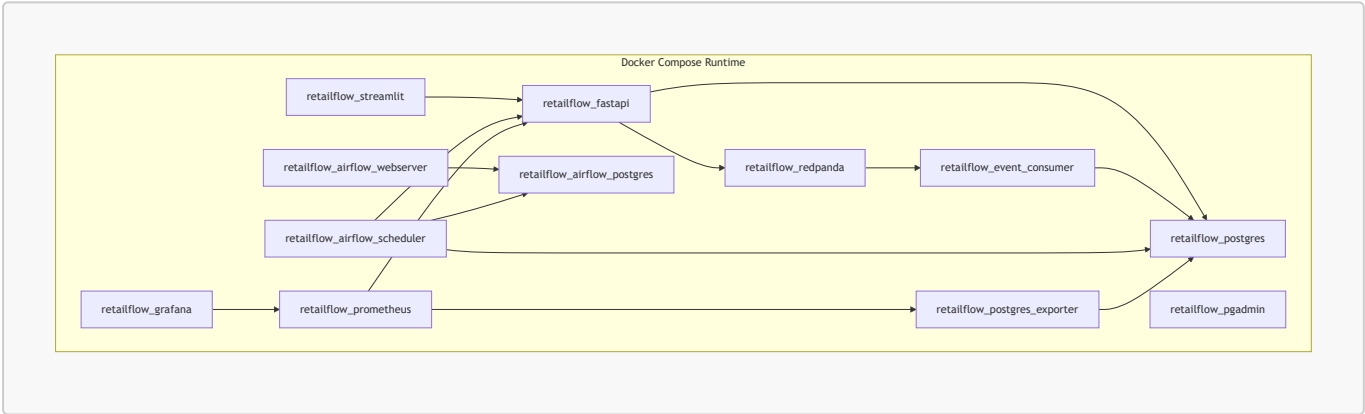
The main services are:

Service	Role
PostgreSQL	Main database
pgAdmin	Database administration interface
Redpanda	Kafka-compatible event broker
FastAPI	Backend service layer

Service	Role
Event consumer	Streaming consumer and validation process
Streamlit	Platform user interface
Airflow webserver	Orchestration UI
Airflow scheduler	DAG scheduling
Airflow PostgreSQL	Airflow metadata database
Prometheus	Metrics collection
Grafana	Metrics visualization
PostgreSQL exporter	Database metrics exporter

6. Docker Compose Architecture

Docker Compose is used as the main local deployment mechanism.



This setup allows the full platform to run with a single command:

```
docker compose up -d
```

7. Service Responsibilities

7.1 PostgreSQL

PostgreSQL is the central data platform.

It stores:

- raw events;
- clean business entities;
- analytical features;
- ML predictions;
- customer segments;

- governance policies;
- retention logs;
- data quality logs;
- dead-letter events.

PostgreSQL provides a stable relational foundation for both operational and analytical data.

7.2 Redpanda

Redpanda is used as the event broker.

It receives customer events from FastAPI and exposes them to the event consumer.

I selected Redpanda because it provides Kafka-compatible streaming capabilities with a simpler local deployment model.

7.3 FastAPI

FastAPI is the backend service layer.

It exposes:

- product endpoints;
- event publishing endpoints;
- recent event endpoints;
- quality endpoints;
- governance endpoints;
- AI endpoints;
- model report endpoints;
- health and metrics endpoints.

FastAPI also produces live events to Redpanda.

7.4 Event Consumer

The event consumer processes messages from Redpanda.

It is responsible for:

- consuming messages;
 - parsing payloads;
 - applying validation rules;
 - writing valid events to PostgreSQL;
 - writing invalid events to dead-letter tables;
 - writing failed quality checks to governance logs.
-

7.5 Streamlit

Streamlit is the product interface.

It provides pages for:

- platform overview;
- customer journey simulation;
- customer intelligence;
- data governance;
- data quality;
- AI monitoring;
- observability.

7.6 Airflow

Airflow orchestrates scheduled workflows.

Main DAGs:

DAG	Schedule	Purpose
daily_sales_aggregation	Daily	Build daily sales aggregates
daily_data_quality	Daily	Check data quality indicators
ml_retraining	Weekly	Retrain ML models and refresh predictions
retention_cleanup	Weekly	Apply retention and anonymization logic

7.7 Prometheus

Prometheus collects metrics from FastAPI and PostgreSQL exporter.

It supports operational monitoring and observability.

7.8 Grafana

Grafana visualizes Prometheus metrics.

It provides operational dashboards for platform health.

7.9 PostgreSQL Exporter

The PostgreSQL exporter exposes database metrics to Prometheus.

It allows monitoring of PostgreSQL availability and operational behavior.

7.10 GitHub Actions

GitHub Actions provides the CI/CD automation layer.

It supports:

- code quality checks;
- automated tests;
- Docker build validation;
- API smoke checks;
- deployment-readiness validation.

The CI/CD layer is part of the platform industrialization strategy.

8. Network and Communication Design

RetailFlow services communicate through the Docker network.

Internal service names are used inside containers.

Examples:

From	To	Internal URL
Streamlit	FastAPI	<code>http://fastapi:8000</code>
FastAPI	PostgreSQL	<code>postgres:5432</code>
FastAPI	Redpanda	<code>redpanda:9092</code>
Event consumer	Redpanda	<code>redpanda:9092</code>
Event consumer	PostgreSQL	<code>postgres:5432</code>
Prometheus	FastAPI	<code>fastapi:8000/metrics</code>
Prometheus	PostgreSQL exporter	<code>postgres_exporter:9187/metrics</code>
Grafana	Prometheus	<code>prometheus:9090</code>

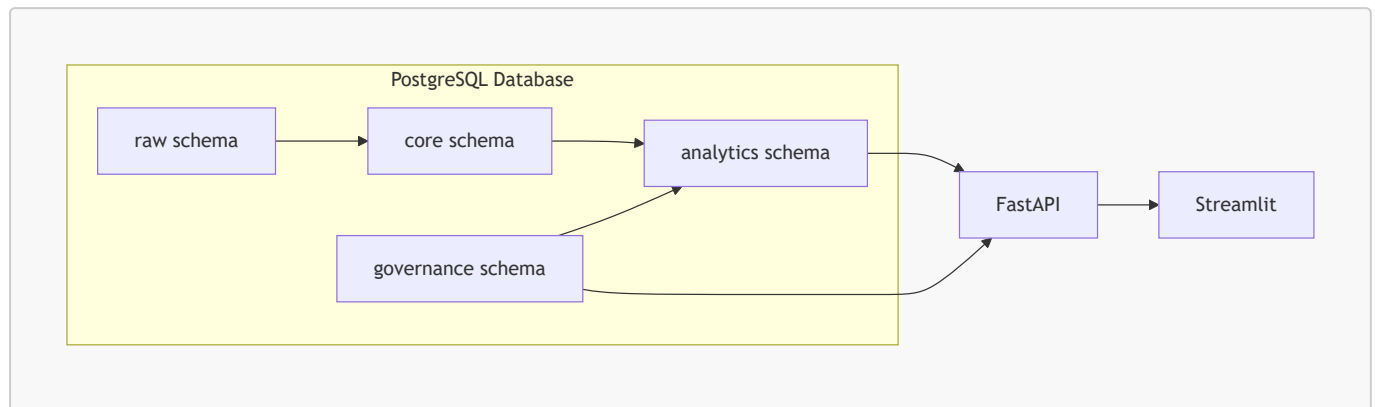
External access uses localhost ports.

Component	Local URL
Streamlit	<code>http://127.0.0.1:8501</code>
FastAPI	<code>http://127.0.0.1:8000</code>
FastAPI Docs	<code>http://127.0.0.1:8000/docs</code>
PostgreSQL	<code>localhost:5432</code>
Airflow	<code>http://127.0.0.1:8080</code>
Prometheus	<code>http://127.0.0.1:9090</code>
Grafana	<code>http://127.0.0.1:3000</code>
PostgreSQL exporter	<code>http://127.0.0.1:9187/metrics</code>

9. PostgreSQL Data Architecture

PostgreSQL is organized into logical schemas.

This schema separation is one of the most important architecture choices.



10. Database Schemas

10.1 Raw Schema

The **raw** schema stores ingested events and raw behavioral information.

Purpose:

- preserve event-level data;
- support replay and traceability;
- keep ingestion data separate from clean analytical entities.

Example table:

```
raw.events
```

10.2 Core Schema

The **core** schema stores business entities.

Examples:

- customers;
- products;
- orders;
- order items;
- payments;
- shipments;
- returns;
- reviews;
- support tickets.

The core schema represents the trusted business layer.

10.3 Analytics Schema

The **analytics** schema stores derived and analytical data.

Examples:

- customer features;
- daily sales aggregates;
- ML predictions;
- customer segments;
- analytical views.

This layer supports dashboards, ML workflows and business intelligence.

10.4 Governance Schema

The **governance** schema stores governance, compliance, retention and quality data.

Examples:

- customer consents;
- data retention policies;
- retention action logs;
- data quality logs;
- dead-letter events.

This schema makes governance observable and auditable.

11. Data Modeling Architecture

The RetailFlow data model is designed to support both operational retail workflows and analytical decision-making.

The model follows a layered approach:

```
raw event data
→ trusted business entities
→ analytical features and aggregates
→ machine learning predictions
→ governed business dashboards
```

This design gives the platform a clear separation between:

- raw ingestion;
- operational truth;

- analytical consumption;
- governance and auditability;
- AI-ready features.

The data model is customer-centric because the main business questions of RetailFlow are centered on customer behavior, customer value, churn risk, consent, lifecycle and segmentation.

11.1 Conceptual Data Domains

The data model is organized into seven business domains.

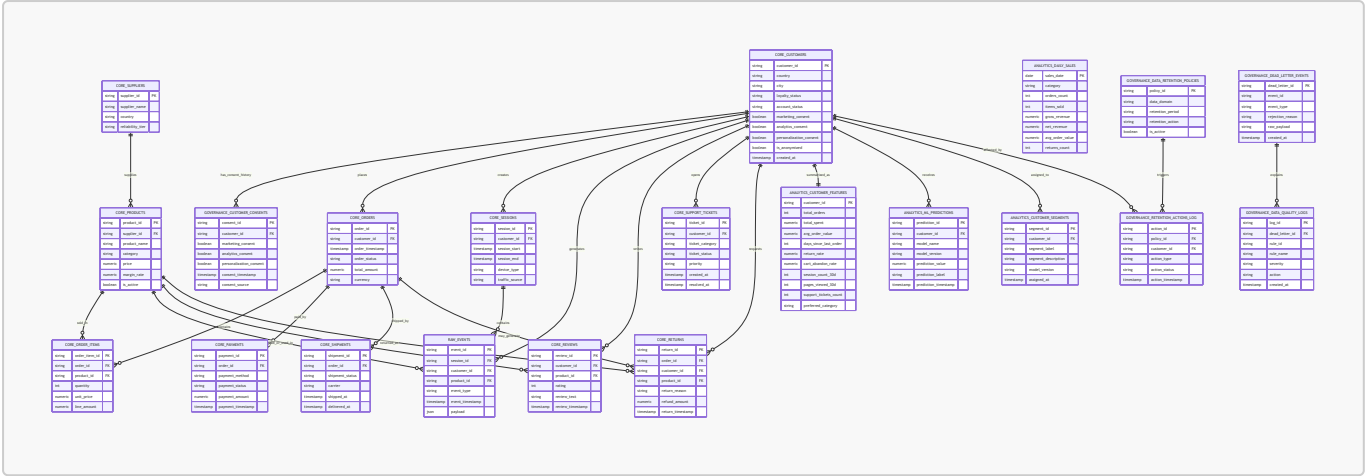
Domain	Main Tables	Purpose
Customer domain	<code>core.customers</code> , <code>governance.customer_consent</code> s	Identify customers, statuses and consent context
Product domain	<code>core.products</code> , <code>core.suppliers</code>	Manage catalog and supplier relationships
Commerce domain	<code>core.orders</code> , <code>core.order_items</code> , <code>core.payments</code> , <code>core.shipments</code> , <code>core.returns</code>	Represent the complete transaction lifecycle
Interaction domain	<code>raw.events</code> , <code>core.sessions</code> , <code>core.reviews</code> , <code>core.support_tickets</code>	Capture behavioral and service interactions
Analytics domain	<code>analytics.customer_features</code> , <code>analytics.daily_sales</code>	Store derived indicators and BI-ready aggregates
AI domain	<code>analytics.ml_predictions</code> , <code>analytics.customer_segments</code>	Store model outputs, customer intelligence and segmentation
Governance domain	<code>governance.data_retention_policies</code> , <code>governance.retention_actions_log</code> , <code>governance.dead_letter_events</code> , <code>governance.data_quality_logs</code>	Support privacy, retention, auditability and data quality

This domain structure allows RetailFlow to cover operational, analytical, governance and AI use cases in a single coherent PostgreSQL architecture.

11.2 Complete Implemented Entity Relationship Diagram

The following ERD describes the complete implemented RetailFlow relational model.

The diagram uses logical entity names to make the relationships readable. The physical implementation is organized across PostgreSQL schemas: `raw`, `core`, `analytics` and `governance`.



This ERD shows that RetailFlow is not a flat dataset.

It is a normalized and governed relational model that supports:

- customer analytics;
- commerce analytics;
- event streaming;
- quality control;
- AI features;
- retention and consent auditability;
- observability of data failures.

11.3 Relationship Explanation

Relationship	Meaning
customers → orders	A customer can place multiple orders
orders → order_items	Each order contains one or more products
products → order_items	A product can appear in many order lines
orders → payments	An order can have one or more payment records
orders → shipments	An order can be associated with shipment tracking
orders → returns	An order can generate return records
customers → sessions	A customer can create multiple browsing sessions
sessions → events	A session contains a sequence of customer events
customers → events	Events are linked to customers when available
products → events	Product-level events support product analytics
customers → reviews	A customer can write reviews
products → reviews	A product can receive reviews
customers → support_tickets	A customer can open support cases

Relationship	Meaning
customers → customer_features	Customer features summarize behavioral and transactional history
customers → ml_predictions	AI predictions are generated at customer level
customers → customer_segments	Segments assign customers to business-readable groups
customers → customer_consent	Consent history is attached to the customer profile
retention_policies → retention_actions_log	Retention actions are executed according to governance policies
dead_letter_events → data_quality_logs	Rejected events are linked to the quality rules that failed

11.4 Core Entity Descriptions

Customers

core.customers is the central master entity of the platform.

It stores customer identity, geographic context, loyalty status, account status and consent flags.

Primary use cases:

- customer portfolio analysis;
- churn prediction;
- CLV prediction;
- consent-aware analytics;
- retention and anonymization workflows.

Primary key:

customer_id

Main relationships:

customers → orders
customers → sessions
customers → events
customers → customer_features
customers → ml_predictions
customers → customer_segments
customers → customer_consent

Products

core.products stores the product catalog.

It connects sales, events, reviews, returns and supplier relationships.

Primary use cases:

- sales analysis;
- category performance;
- product interaction tracking;
- return analysis;
- product-level customer journey analysis.

Primary key:

```
product_id
```

Main relationships:

```
products → order_items  
products → reviews  
products → returns  
products → events  
suppliers → products
```

Orders and Order Items

core.orders represents the order header.

core.order_items represents product-level order lines.

This split is important because one order can contain several products.

Primary use cases:

- revenue analytics;
- average order value;
- product sales performance;
- customer purchase history;
- CLV feature engineering.

Primary keys:

```
order_id  
order_item_id
```


Main relationships:

```
customers → orders
orders → order_items
products → order_items
orders → payments
orders → shipments
orders → returns
```

Payments

`core.payments` stores payment information attached to orders.

Primary use cases:

- payment validation;
- payment status tracking;
- transaction completeness checks;
- revenue reconciliation.

Shipments

`core.shipments` stores delivery information.

Primary use cases:

- delivery monitoring;
- fulfillment tracking;
- operational analysis;
- customer experience analysis.

Returns

`core.returns` stores returned products and refund information.

Primary use cases:

- return rate computation;
- customer dissatisfaction signals;
- product quality analysis;
- churn feature engineering.

Sessions and Events

`core.sessions` describes browsing sessions.

`raw.events` stores event-level customer activity.

This separation supports both session-level analysis and event-level replay.

Primary use cases:

- behavioral analytics;
 - cart abandonment analysis;
 - product view analysis;
 - funnel tracking;
 - real-time pipeline validation.
-

Reviews and Support Tickets

`core.reviews` captures customer feedback on products.

`core.support_tickets` captures service interactions.

Primary use cases:

- satisfaction analysis;
 - customer experience monitoring;
 - support load analysis;
 - churn risk feature engineering.
-

Customer Features

`analytics.customer_features` is the customer-level feature table used by machine learning and dashboards.

It combines transactional, behavioral and service signals.

Examples:

- total orders;
 - total spent;
 - days since last order;
 - return rate;
 - cart abandonment rate;
 - support ticket count;
 - preferred category.
-

ML Predictions and Segments

`analytics.ml_predictions` stores churn and CLV outputs.

`analytics.customer_segments` stores segmentation assignments.

These tables make the ML layer queryable and auditable through SQL and FastAPI.

Governance Tables

The governance schema contains:

- consent history;
- retention policies;
- retention action logs;
- dead-letter events;
- data quality logs.

These entities make governance measurable and traceable.

12. Implemented Star Schema Architecture

The RetailFlow relational model is normalized for consistency and governance.

On top of this normalized model, I implemented analytical structures that behave like star schemas for reporting, dashboards and AI features.

In this document, I use analytical names such as `fact_sales`, `fact_customer_activity` and `fact_ai_predictions` to describe the star schema design.

In the current PostgreSQL implementation, these fact structures are represented by existing physical tables and aggregates:

Analytical Fact Name	Implemented Physical Representation
<code>fact_sales</code>	<code>core.orders</code> , <code>core.order_items</code> , <code>core.payments</code> , <code>core.returns</code> , <code>analytics.daily_sales</code>
<code>fact_customer_activity</code>	<code>raw.events</code> , <code>core.sessions</code> , <code>analytics.customer_features</code>
<code>fact_ai_predictions</code>	<code>analytics.ml_predictions</code> , <code>analytics.customer_segments</code> , <code>analytics.customer_features</code>

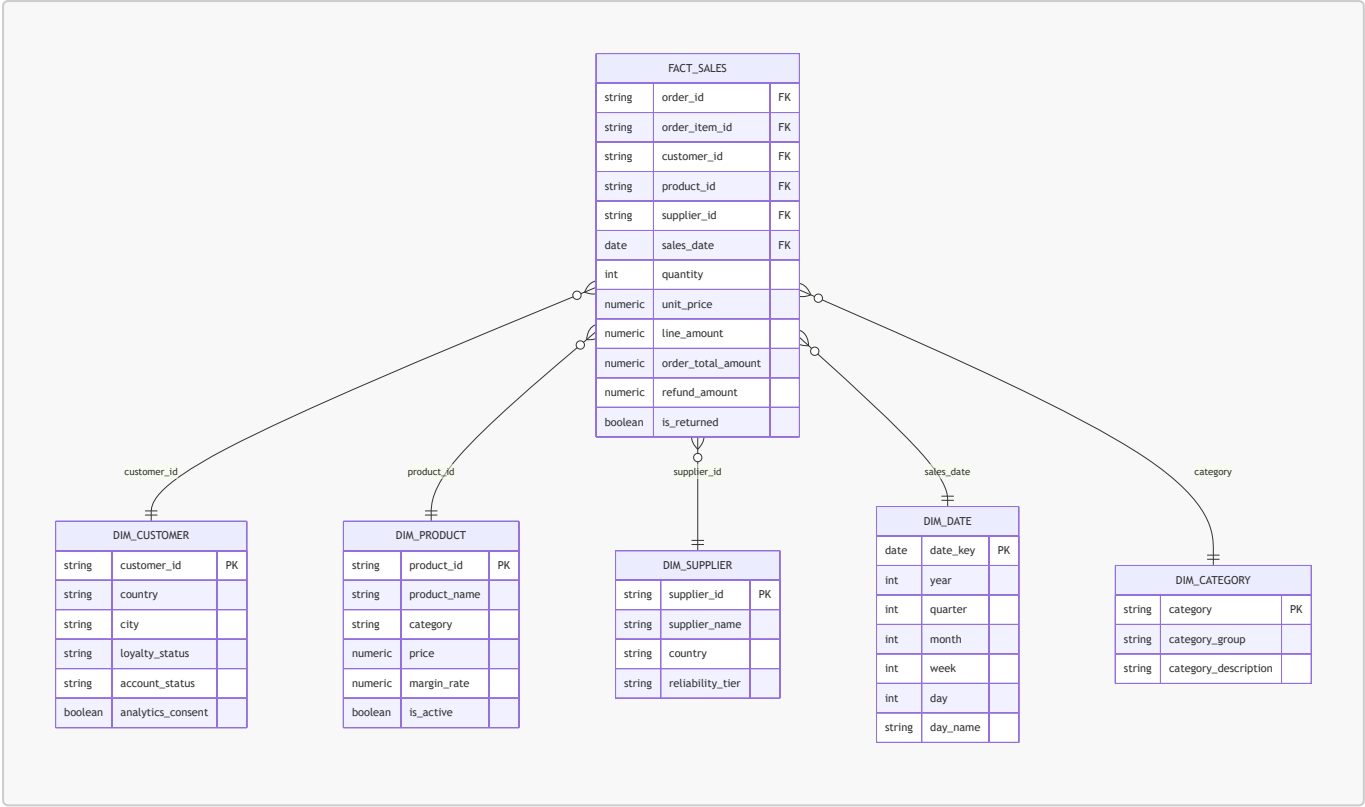
This means that the star schemas are not only theoretical.

They are implemented through the current PostgreSQL tables and used by dashboards, API endpoints and ML workflows.

12.1 Sales Analytics Star Schema

The sales star schema is centered on order and order line transactions.

It supports revenue analysis, category performance, product performance and customer value analysis.



Implementation mapping:

Star Schema Object	Implemented Tables
fact_sales	core.orders, core.order_items, core.payments, core.returns, analytics.daily_sales
dim_customer	core.customers
dim_product	core.products
dim_supplier	core.suppliers
dim_date	derived from order timestamps and aggregation dates
dim_category	product category attribute in core.products

Main measures:

Measure	Meaning
orders_count	Number of orders
items_sold	Number of product units sold
gross_revenue	Revenue before returns
net_revenue	Revenue after returns
avg_order_value	Average value per order
returns_count	Number of returned items or orders

Measure	Meaning
return_rate	Ratio of returns compared with purchases

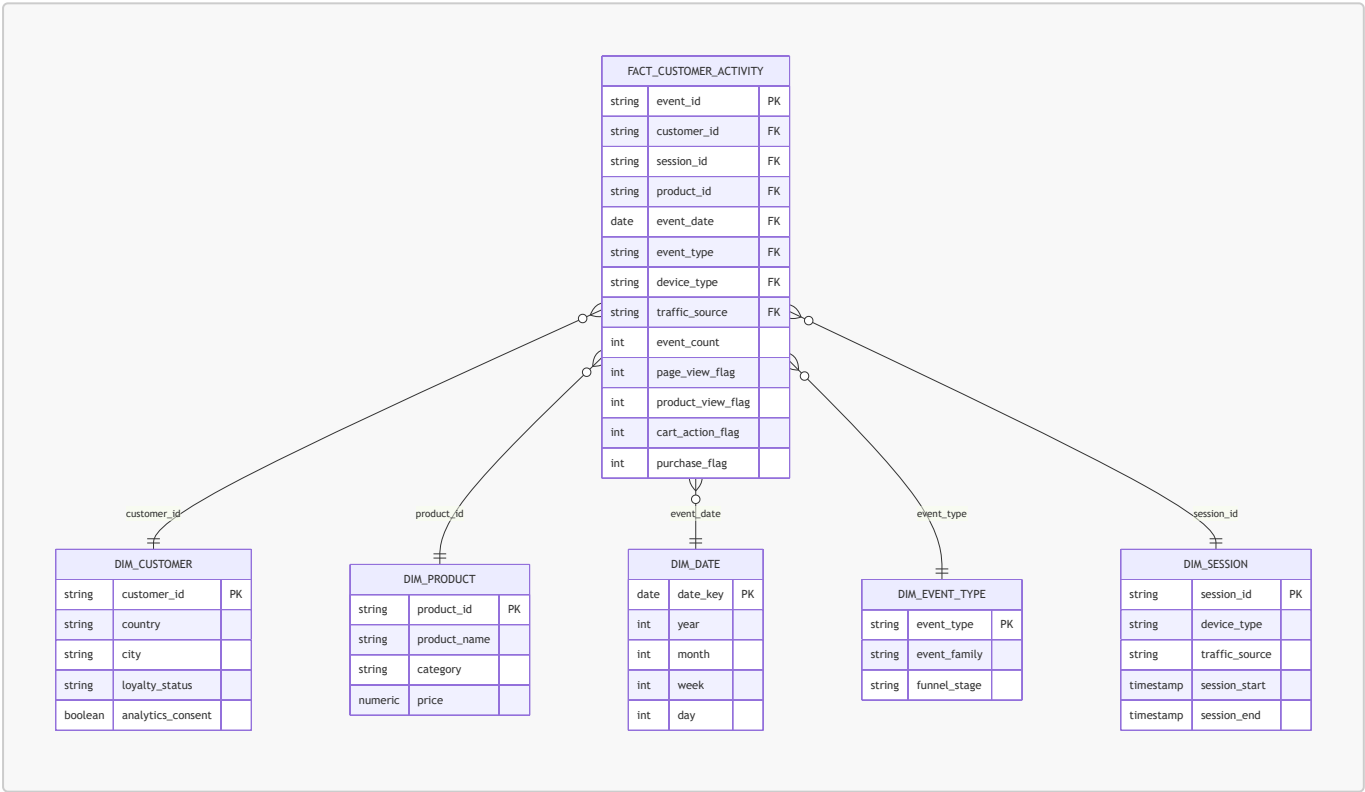
Business questions supported:

- What are the highest revenue categories?
- Which customer groups generate the most sales?
- Which products have high sales but also high returns?
- How does daily revenue evolve?
- Which suppliers contribute most to revenue?

12.2 Customer Activity Star Schema

The customer activity star schema is centered on event-level behavior.

It supports funnel monitoring, product engagement, browsing analysis and behavioral feature engineering.



Implementation mapping:

Star Schema Object	Implemented Tables
fact_customer_activity	raw.events, core.sessions
dim_customer	core.customers
dim_product	core.products
dim_date	derived from event_timestamp
dim_event_type	event type values validated by the consumer

Star Schema Object	Implemented Tables
dim_session	core.sessions

Main measures:

Measure	Meaning
event_count	Number of customer events
session_count	Number of sessions
page_views	Number of page views
product_views	Number of product views
add_to_cart_count	Number of add-to-cart actions
checkout_started_count	Number of checkout starts
purchase_count	Number of purchase events
cart_abandon_rate	Behavioral signal used for customer intelligence

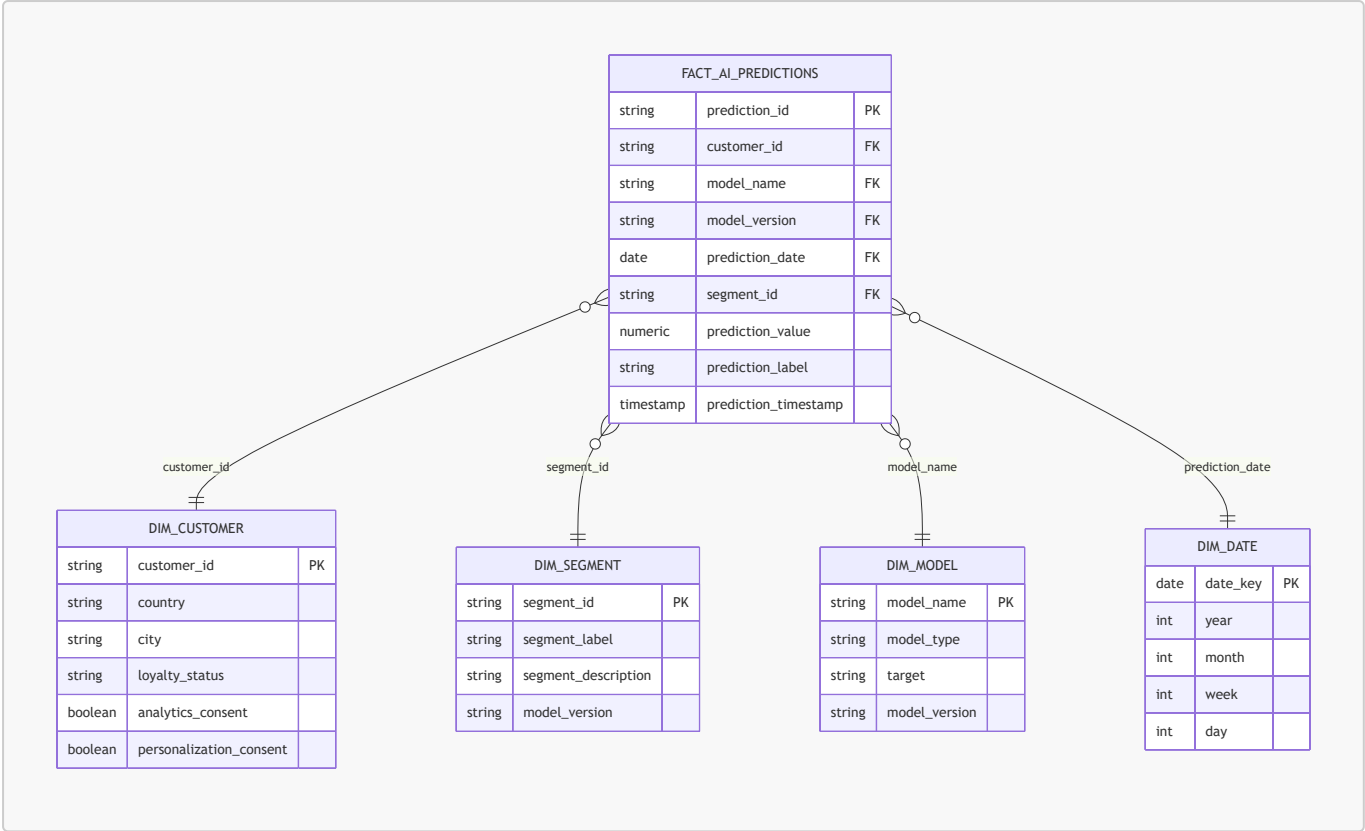
Business questions supported:

- Which event types occur most frequently?
- Where do customers abandon the journey?
- Which products generate views but not purchases?
- Which customers have high activity but low conversion?
- Are incoming events valid and complete?

12.3 AI Predictions Star Schema

The AI predictions star schema is centered on model outputs.

It supports AI monitoring, customer prioritization, segmentation analysis and model explainability.



Implementation mapping:

Star Schema Object	Implemented Tables
fact_ai_predictions	analytics.ml_predictions
dim_customer	core.customers
dim_segment	analytics.customer_segments
dim_model	model metadata from ml/reports/*.json and prediction fields
dim_date	derived from prediction_timestamp

Main measures:

Measure	Meaning
prediction_value	Churn probability or predicted CLV value
predictions_count	Number of generated predictions
avg_prediction_value	Average model output by label or segment
high_risk_customers_count	Number of customers with high churn risk
high_value_customers_count	Number of customers with high CLV band
segment_customers_count	Number of customers assigned to each segment

Business questions supported:

- Which customers have the highest churn risk?

- Which customers have the highest predicted CLV?
- Which segments contain the highest-value customers?
- Are predictions distributed consistently?
- Which model version generated each prediction?

12.4 Dimension Strategy

The dimension layer is designed to make analytics readable and reusable.

Dimension	Source	Purpose
dim_customer	core.customers	Customer segmentation, geography, loyalty and consent filtering
dim_product	core.products	Product and category analysis
dim_date	timestamps in operational and analytical tables	Time-based reporting
dim_category	core.products.category	Category-level performance
dim_country	core.customers.country, core.suppliers.country	Geographic analysis
dim_supplier	core.suppliers	Supplier performance
dim_segment	analytics.customer_segments	Segment-level customer intelligence
dim_event_type	event validation rules and raw.events.event_type	Funnel and behavioral analysis
dim_model	ML reports and prediction metadata	Model monitoring and version traceability

This strategy avoids duplicating business definitions across dashboards and APIs.

12.5 Granularity and Analytical Grain

Each fact structure has a clear grain.

Fact	Grain	Why It Matters
fact_sales	one order item / sales line	Allows product, customer, category and supplier analysis
fact_customer_activity	one customer event	Allows funnel analysis and behavioral monitoring
fact_ai_predictions	one model prediction per customer, model and timestamp	Allows model versioning, monitoring and traceability

Defining grain is important because it prevents ambiguous metrics.

For example:

- revenue belongs to sales facts;
- event counts belong to activity facts;
- churn probability belongs to AI prediction facts.

This separation makes dashboards easier to validate and maintain.

12.6 Star Schema Usage in RetailFlow

The star schema design is used by several platform components.

Component	Star Schema Usage
Streamlit Platform Overview	sales, customers, events and monitoring indicators
Streamlit Customer Intelligence	customer features, predictions and segments
Streamlit Data Quality	activity validation and rejected event analysis
Streamlit AI Monitoring	AI prediction and model report analysis
FastAPI <code>/analytics</code>	aggregated analytical data
FastAPI <code>/ai</code>	predictions, segments and model reports
Airflow <code>daily_sales_aggregation</code>	sales fact aggregation
Airflow <code>ml_retraining</code>	feature generation and prediction refresh

13. Data Modeling Design Decisions

13.1 Normalized Core Model

The `core` schema is normalized to preserve data consistency.

This is important because orders, products, payments, shipments, returns and customers have different lifecycles.

For example:

```
one order
→ many order items
→ one product per order line
```

A flat table would duplicate product and customer attributes across order lines.

The normalized model avoids this duplication.

13.2 Analytical Layer for Decision-Making

The **analytics** schema stores derived data that is optimized for analysis and ML consumption.

Examples:

- **analytics.customer_features;**
- **analytics.daily_sales;**
- **analytics.ml_predictions;**
- **analytics.customer_segments.**

This avoids forcing dashboards and ML scripts to recompute expensive metrics from raw transactional tables every time.

13.3 Governance Layer for Accountability

The **governance** schema stores consent, retention, quality and audit records.

This design supports privacy and compliance use cases directly in the data architecture.

Examples:

```
governance.customer_consent  
governance.data_retention_policies  
governance.retention_actions_log  
governance.dead_letter_events  
governance.data_quality_logs
```

13.4 Event Layer for Traceability

The **raw** schema stores customer events.

This layer supports:

- replay;
- debugging;
- behavioral analysis;
- feature engineering;
- data quality investigation.

Valid events are stored in **raw.events**.

Invalid events are isolated in **governance.dead_letter_events**.

13.5 AI-Ready Modeling

The data model is designed to support machine learning without requiring a separate data platform.

Customer intelligence is built from:

```
core.customers
core.orders
core.order_items
core.returns
core.reviews
core.support_tickets
core.sessions
raw.events
analytics.customer_features
```

Model outputs are stored back into PostgreSQL:

```
analytics.ml_predictions
analytics.customer_segments
```

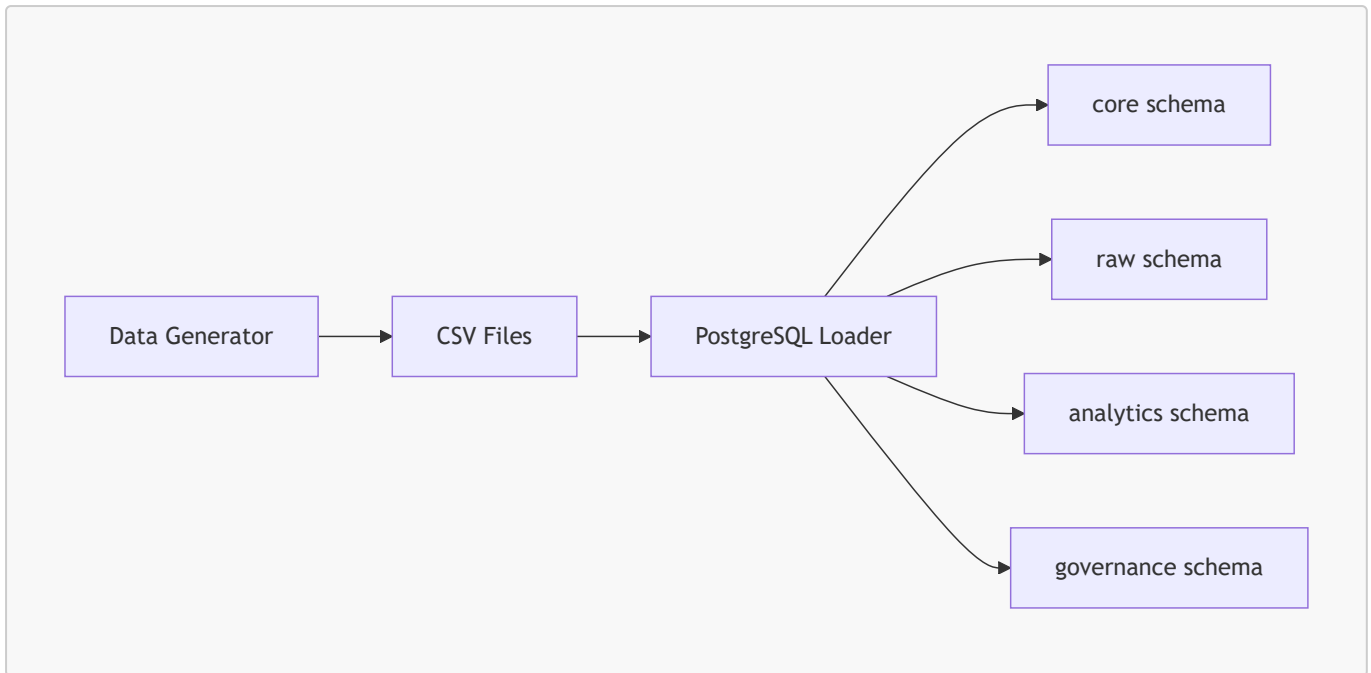
This makes AI outputs accessible through SQL, FastAPI and Streamlit.

13.6 Architecture Objectives Achieved by the Data Model

Objective	Data Modeling Response
Support operational retail data	Normalized core schema
Support event-driven ingestion	raw.events and streaming consumer
Support analytics	analytics.daily_sales and customer features
Support AI	ML features, predictions and segments
Support governance	consent, retention, quality and dead-letter tables
Support auditability	retention logs and quality logs
Support dashboarding	star-schema-oriented facts and dimensions
Support scalability	separated domains and reusable analytical tables

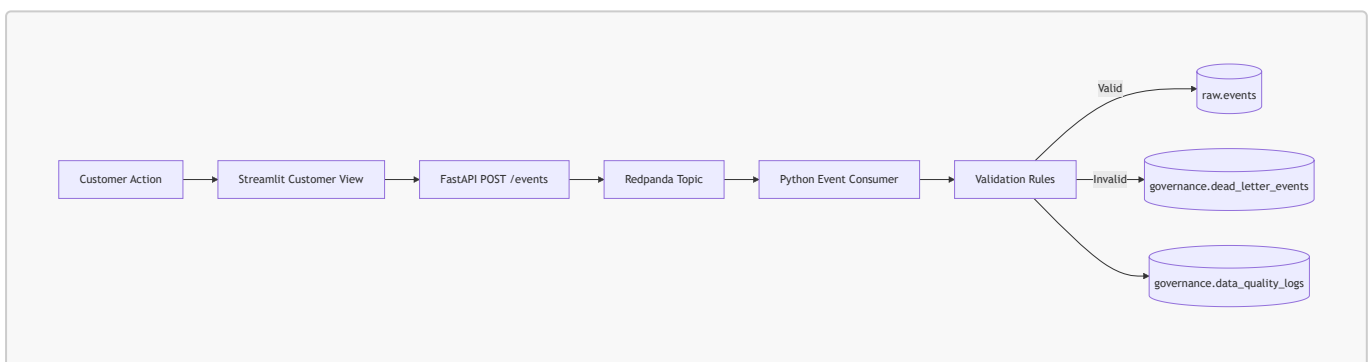
14. Data Flow Architecture

13.1 Historical Data Flow



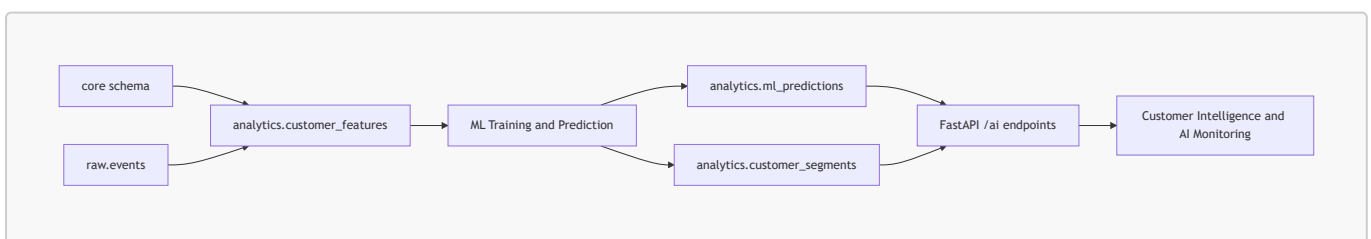
This flow initializes the platform with structured retail data.

13.2 Real-Time Event Flow



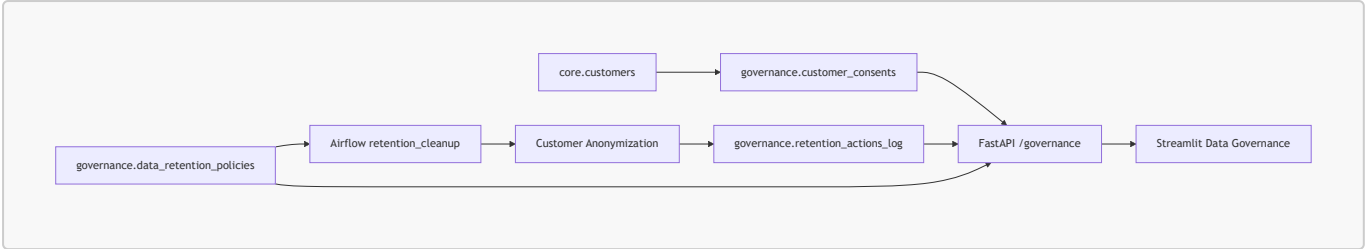
This flow captures customer events and validates them before persistence.

13.3 Analytics Flow



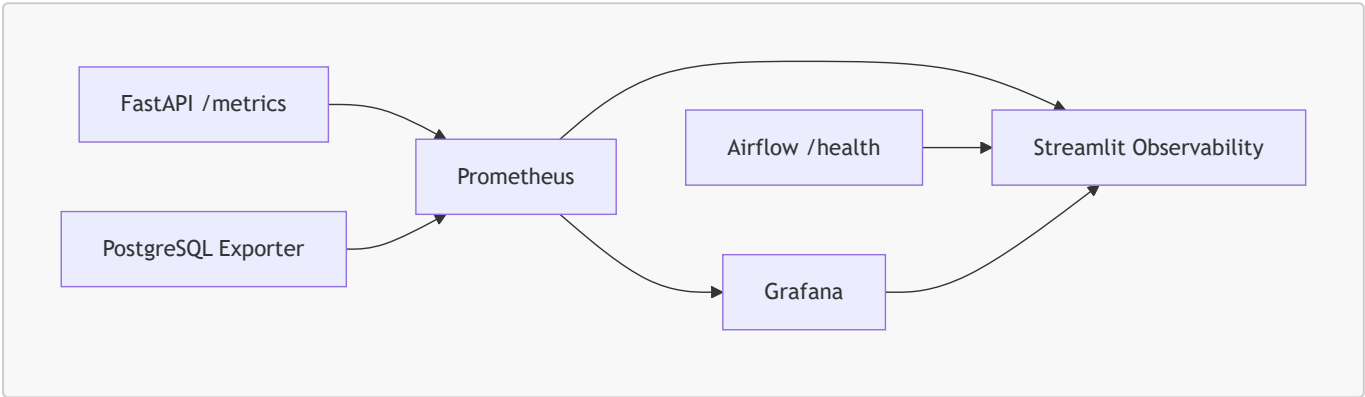
This flow turns customer and event data into AI-driven customer intelligence.

13.4 Governance Flow



This flow shows how governance data becomes visible and auditable.

13.5 Observability Flow



This flow monitors the operational state of the platform.

15. API Architecture

FastAPI is the central access layer for Streamlit and external clients.



Syntax error in text

mermaid version 11.15.0

Main endpoint groups:

Endpoint group	Purpose
/products	Product catalog and product details
/events	Event publishing and recent events
/quality	Dead letters and quality summaries
/governance	Consent, retention and audibility
/ai	Predictions, segments and ML reports
/health	API and database status

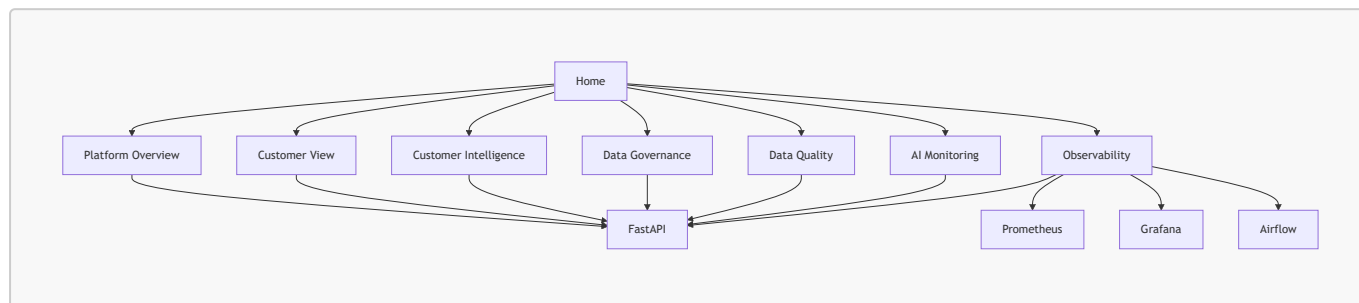
Endpoint group Purpose

/metrics

Prometheus metrics

16. Streamlit Architecture

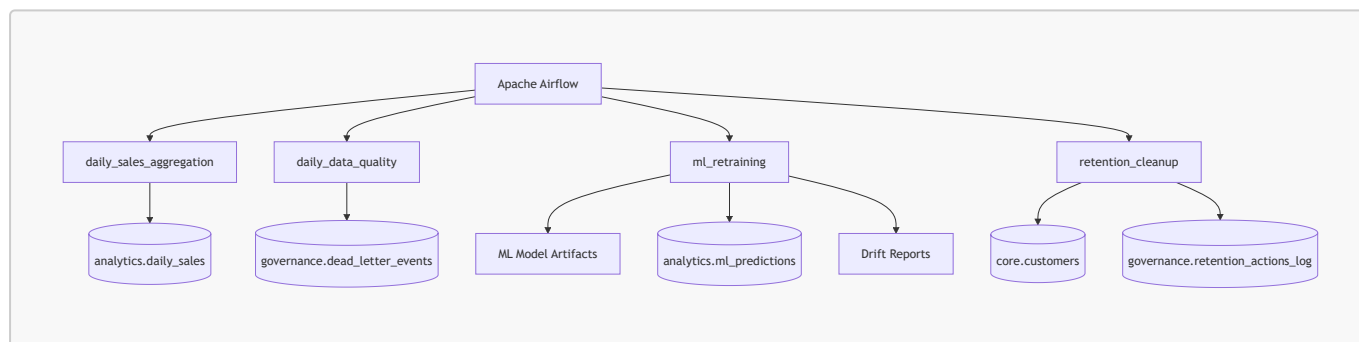
Streamlit is structured as a multi-page platform.



This UI architecture supports both business storytelling and technical proof.

17. Orchestration Architecture

Airflow is used to orchestrate recurring platform workflows.



18. Airflow DAG Design

17.1 daily_sales_aggregation

I implemented the **daily_sales_aggregation** DAG to refresh daily sales indicators.

The DAG:

- reads orders and returns;
- computes daily revenue;
- computes average order value;
- counts returns;
- updates **analytics.daily_sales**;
- validates that the aggregate table contains rows.

This DAG supports the analytics layer.

17.2 daily_data_quality

I implemented the `daily_data_quality` DAG to verify platform quality signals.

The DAG:

- checks PostgreSQL connectivity;
- counts records in `governance.dead_letter_events`;
- provides a scheduled data quality control point.

This DAG supports the quality monitoring strategy.

17.3 ml_retraining

I implemented the `ml_retraining` DAG to orchestrate the ML lifecycle.

It runs:

1. churn model training;
2. segmentation model training;
3. CLV model training;
4. prediction refresh;
5. lightweight drift evaluation.

The dependency structure is:

```
[train_churn, train_segmentation, train_clv]
→ refresh_predictions
→ evaluate_drift
```

This DAG supports the MLOps architecture.

17.4 retention_cleanup

I implemented the `retention_cleanup` DAG to support governance and GDPR-aligned retention.

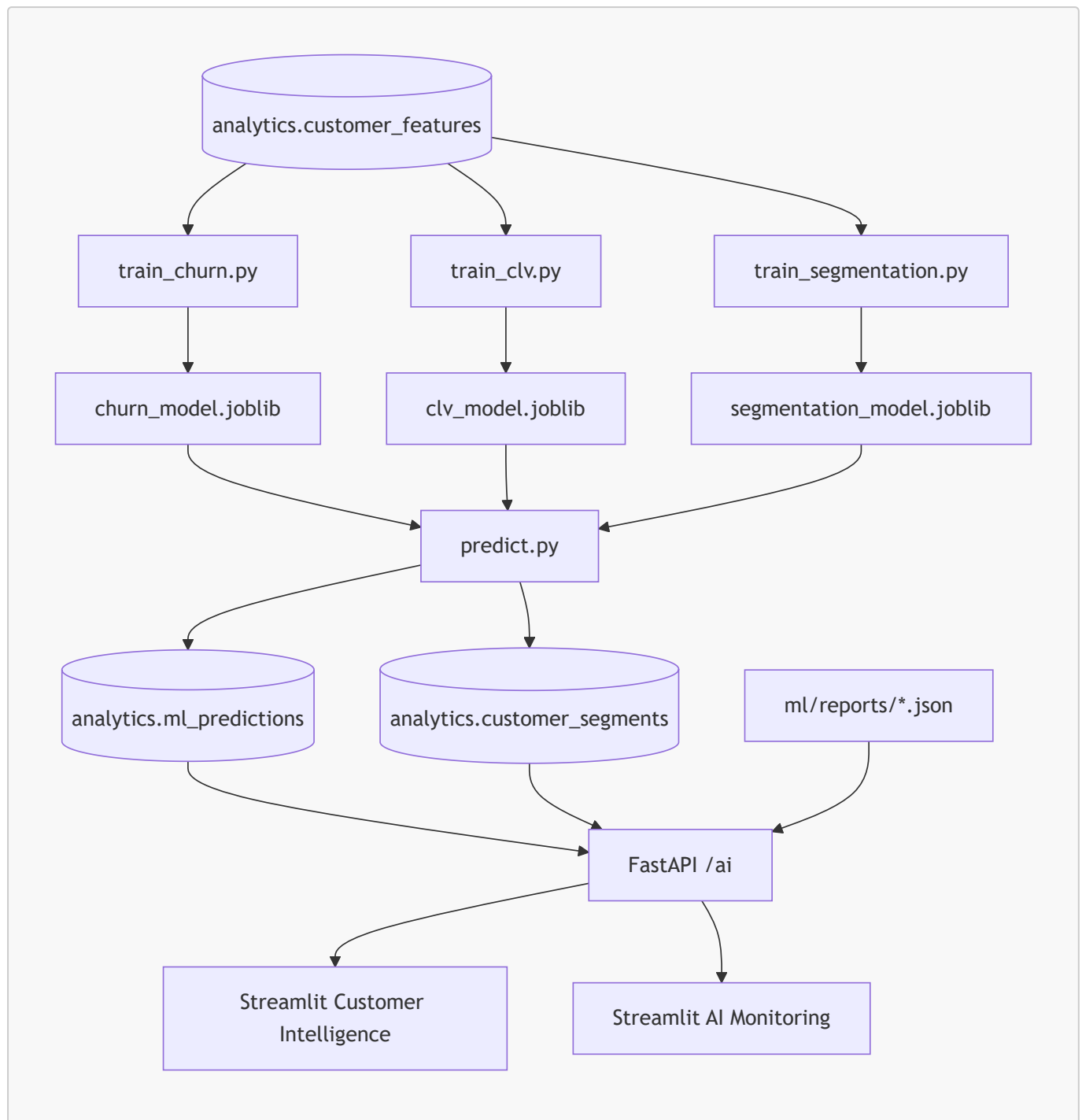
The DAG:

- identifies expired customer records based on retention policy `ret_001`;
- anonymizes personal attributes;
- disables marketing, analytics and personalization consent flags;
- updates the customer status;
- inserts an audit record into `governance.retention_actions_log`.

This DAG is a central proof that governance policies are operationalized.

19. Machine Learning Architecture

The ML architecture is integrated with the data platform.



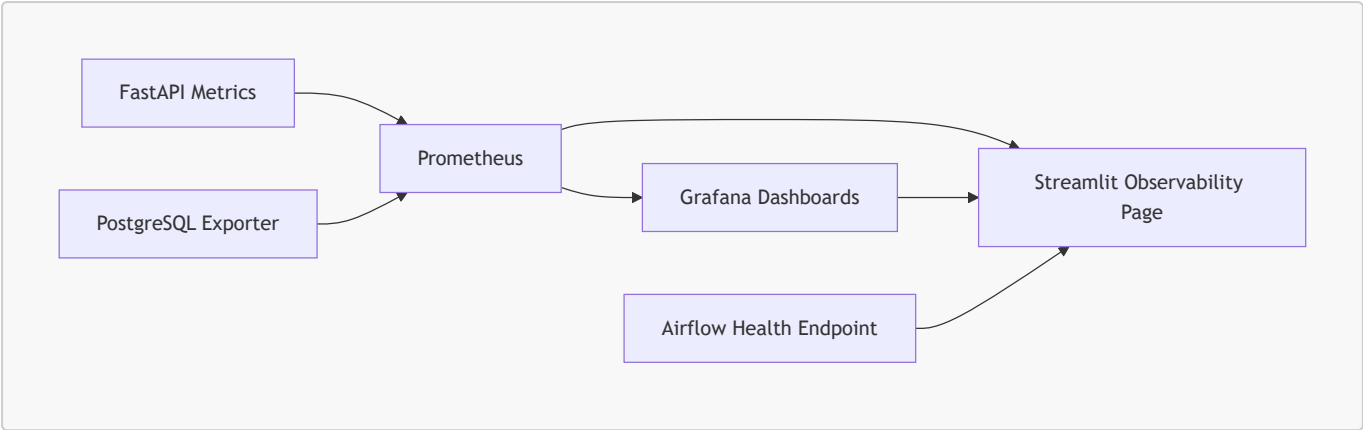
ML is not isolated in notebooks.

It is integrated into:

- storage;
- orchestration;
- API serving;
- monitoring;
- business dashboards.

20. Observability Architecture

RetailFlow includes service and database observability.



Main monitoring assets:

Asset	Purpose
/metrics	FastAPI Prometheus metrics
pg_up	PostgreSQL availability metric
Prometheus targets	Service scrape status
Grafana dashboard	Visual platform monitoring
Alert rules documentation	Operational alert definitions
Streamlit Observability	Consolidated health view

21. Monitoring and Alerting

The architecture includes documented alerting rules.

Main alert categories:

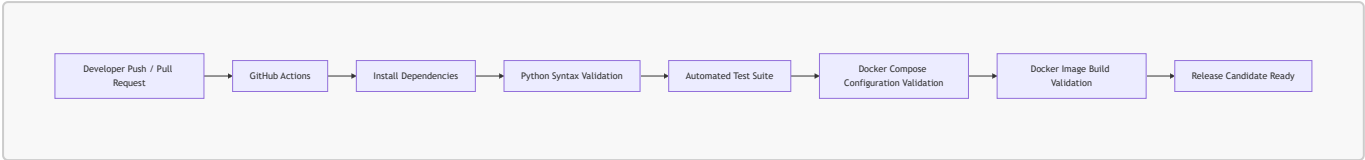
Alert	Purpose
FastAPI Down	Detect API unavailability
PostgreSQL Down	Detect database unavailability
High API Error Rate	Detect server-side API errors
High API Latency	Detect performance degradation
Drift Detected	Connect ML monitoring to operational alerting

These alerts show a production-oriented approach to platform operations.

22. CI/CD Architecture

I implemented GitHub Actions as the CI/CD automation layer.

The CI/CD pipeline validates the platform before changes are integrated into the main development branch.



The implemented CI/CD pipeline includes:

Stage	Purpose
Dependency installation	Install project, API, ML, pipeline and Streamlit dependencies
Python syntax validation	Compile Python modules to detect syntax errors
Automated tests	Run API, data quality and ML artifact tests
Docker Compose validation	Validate the deployment configuration
Docker image build validation	Build FastAPI, Streamlit and event consumer images

The current CI/CD workflow validates:

```
.github/workflows/ci.yml
api/Dockerfile
streamlit_app/Dockerfile
pipeline/consumer/Dockerfile
docker-compose.yml
tests/test_api.py
tests/test_data_quality.py
tests/test_ml.py
```

This creates a deployment-readiness gate.

At the current stage, the pipeline validates the application, tests and container builds.

The remaining production evolution is automated deployment to a target environment such as Kubernetes or a managed cloud platform.

23. Infrastructure as Code Approach

RetailFlow uses Docker Compose as the main infrastructure definition.

The infrastructure is described through:

- `docker-compose.yml`;
- Dockerfiles for FastAPI, Streamlit and consumer services;
- Prometheus configuration;
- Grafana provisioning;
- Airflow DAG definitions;
- database initialization scripts;
- GitHub Actions workflows.

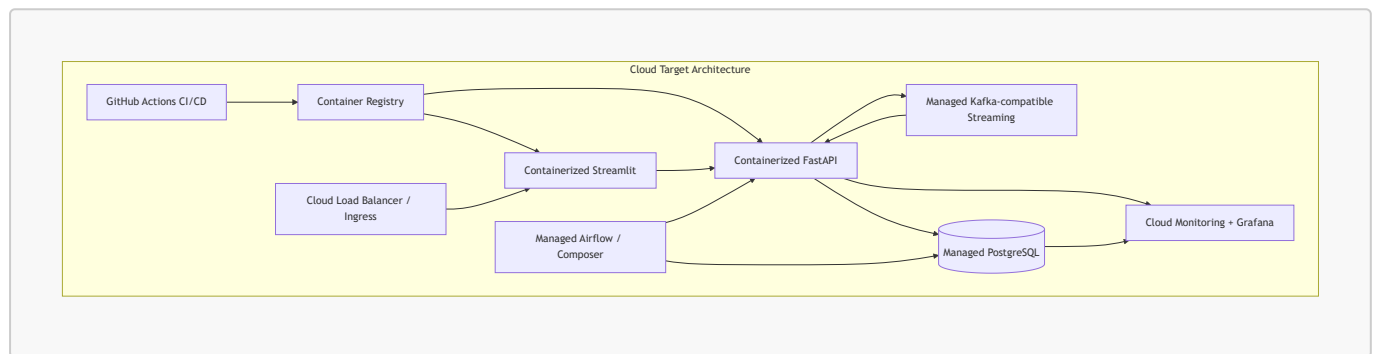
Future infrastructure expansion can include:

- Kubernetes manifests;
- Terraform modules;
- cloud-managed services;
- secrets management;
- cloud monitoring integrations.

24. Cloud Target Architecture

The current platform is local and containerized.

However, the architecture can evolve toward a cloud deployment.



Potential migration strategy:

1. container registry integration;
2. managed PostgreSQL;
3. managed streaming;
4. deployed API and UI services;
5. managed Airflow;
6. cloud-native monitoring;
7. enterprise IAM and SSO;
8. multi-region deployment.

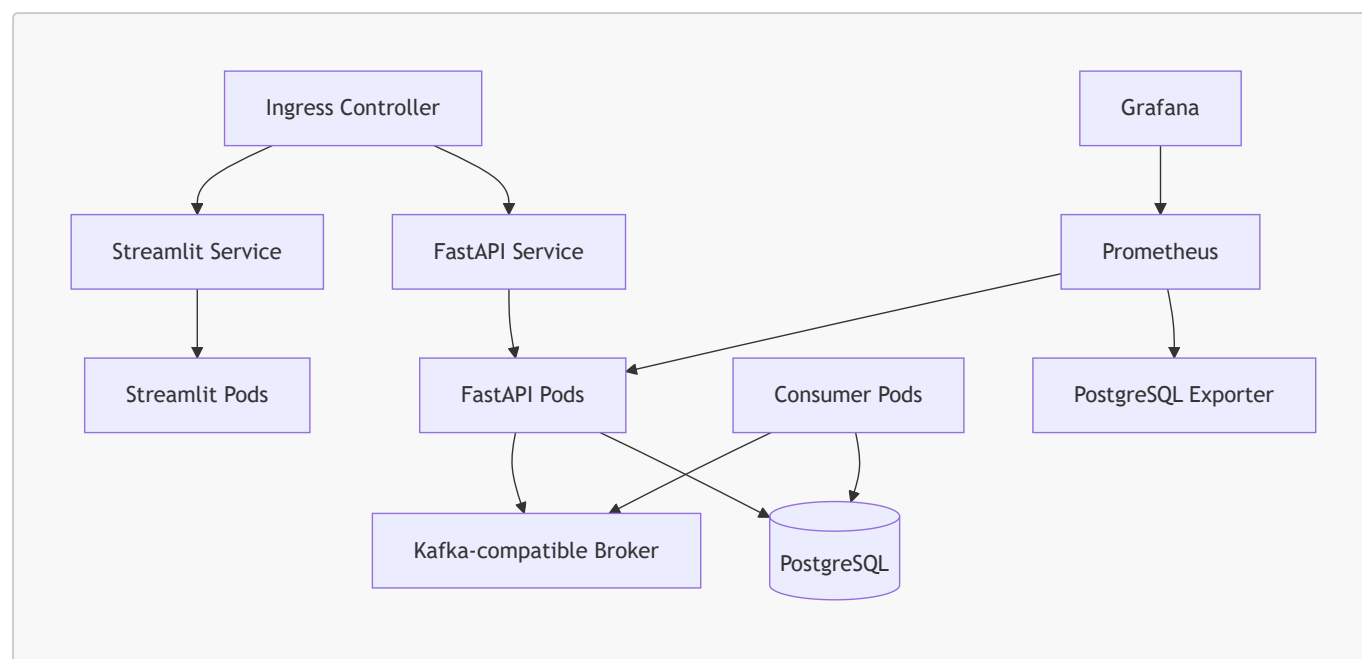
25. Kubernetes Roadmap

The `/k8s` folder is part of the future deployment roadmap.

Kubernetes can be used to deploy:

- FastAPI deployment;
- Streamlit deployment;
- event consumer deployment;
- service manifests;
- ConfigMaps;
- Secrets;
- Ingress;
- horizontal scaling rules.

Future Kubernetes architecture:



Kubernetes is not the primary runtime for the current stable version.

It is part of the scalability and production-readiness roadmap.

26. Security Architecture

RetailFlow includes several security-oriented design decisions.

Current controls:

Control	Description
Environment variables	Configuration separated from code
Docker network isolation	Services communicate inside a controlled network
Consent-based analytics	Customer intelligence can be filtered by analytics consent
Retention and anonymization	Personal attributes can be anonymized through Airflow
Audit logs	Retention actions are logged
Dead-letter isolation	Invalid data is isolated from trusted tables
Observability	Service health is monitored

Future security controls:

- API authentication;
- role-based access control;
- enterprise IAM;
- SSO;
- secrets management;
- encryption policy;

- audit logging for API access;
- network segmentation;
- vulnerability scanning.

27. Data Governance Integration

Data architecture and governance are tightly connected.

Governance is not external to the architecture.

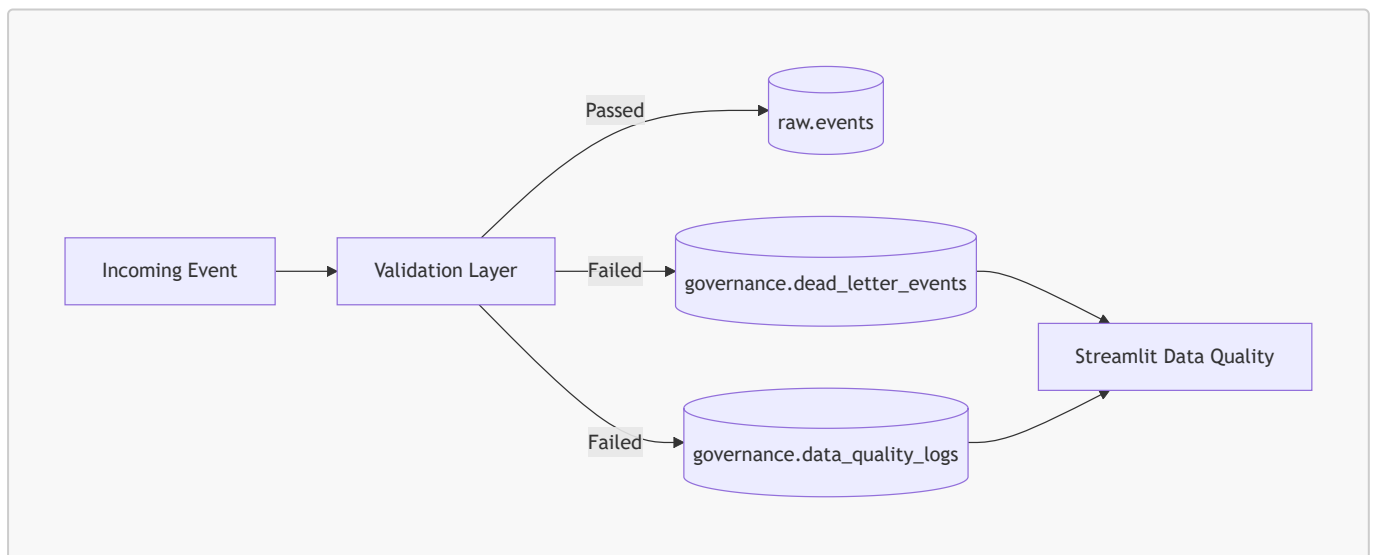
It is implemented through:

- governance schema;
- consent data;
- retention policies;
- anonymization workflow;
- retention logs;
- data quality logs;
- dead-letter tables;
- governed customer intelligence interface.

This design ensures that the architecture supports compliance and accountability.

28. Data Quality Integration

Data quality is integrated into the pipeline architecture.



Invalid events are not ignored.

They are:

- rejected;
- stored;
- categorized;
- made visible;

- available for audit and debugging.

29. Architecture Trade-Offs

28.1 Docker Compose vs Kubernetes

I used Docker Compose as the main runtime because it provides strong local reproducibility.

Docker Compose	Kubernetes
Easier local setup	Better production scaling
Lower complexity	More operational control
Faster iteration	Better enterprise deployment
Suitable for demonstration	Suitable for production clusters

Current choice:

```
Docker Compose for local deployment and evaluation.
```

Future path:

```
Kubernetes for production-grade deployment.
```

28.2 PostgreSQL vs Distributed Warehouse

PostgreSQL was chosen because it is:

- reliable;
- simple to run locally;
- SQL-native;
- suitable for structured retail data;
- compatible with analytics and governance use cases.

A distributed warehouse could be considered later for larger scale.

28.3 Redpanda vs Kafka

Redpanda was chosen because it is Kafka-compatible but easier to run locally.

It preserves the streaming design pattern while reducing deployment complexity.

28.4 Streamlit vs Custom Frontend

Streamlit was chosen because it enables fast development of analytical and monitoring dashboards.

A custom frontend could be considered later for a production SaaS experience.

28.5 FastAPI vs Flask

FastAPI was chosen because it provides:

- automatic OpenAPI documentation;
- strong performance;
- modern Python typing;
- clean API structure;
- easy monitoring integration.

30. Architecture Evaluation Against Requirements

Requirement	Architecture Response
Identify technical needs	Architecture covers streaming, storage, ML, governance and monitoring
Design a complete infrastructure	Docker Compose deploys all main services
Define data models	PostgreSQL schemas separate raw, core, analytics and governance layers
Design database structures	Tables support retail operations, analytics, ML and governance
Deploy server infrastructure	Docker Compose deploys multi-service local platform
Set up monitoring tools	Prometheus, Grafana and PostgreSQL exporter are integrated
Document architecture clearly	README and architecture documents include diagrams and explanations
Support observability	FastAPI metrics, DB metrics and Airflow health are visible
Support automation	Airflow and GitHub Actions provide workflow automation

31. Architecture Strengths

The main strengths of the architecture are:

1. end-to-end integration;
2. clear service separation;
3. governed storage model;
4. real-time event capability;
5. ML and API integration;
6. operational observability;
7. orchestration through Airflow;

- 8. CI/CD through GitHub Actions;
- 9. strong demonstration flow;
- 10. future cloud and Kubernetes roadmap.

32. Architecture Limitations

The current architecture does not yet include:

- enterprise IAM;
- SSO;
- multi-region deployment;
- 24/7 production support and on-call operations;
- fully automated data lineage;
- managed cloud infrastructure;
- enterprise data catalog;
- production-grade secrets management.

These limitations are expected at the current platform stage and are addressed in the future roadmap.

33. Architecture Roadmap

32.1 Completed Architecture Capabilities

I have already implemented:

- Docker Compose multi-service infrastructure;
- PostgreSQL schema separation;
- Redpanda event streaming;
- FastAPI service layer;
- Python event consumer;
- Airflow orchestration;
- Streamlit dashboards;
- Prometheus metrics;
- Grafana dashboards;
- PostgreSQL exporter;
- GitHub Actions CI/CD;
- governance schema;
- AI serving layer;
- ML monitoring dashboards.

32.2 Future Improvements

Planned architecture improvements:

Area	Improvement
Cloud deployment	Deploy services to managed cloud infrastructure

Area	Improvement
Kubernetes	Build production manifests and deployment strategy
IAM	Add enterprise authentication and authorization
Data catalog	Add searchable metadata and ownership catalog
Lineage	Add automated lineage across pipelines and transformations
Model registry	Add formal model versioning and promotion workflow
Secrets management	Move secrets to a dedicated secret manager
Scaling	Add horizontal scaling for API and consumer services
Alerting	Add notification channels and runbooks
Security	Add vulnerability scanning and API authorization

34. Conclusion

I designed the RetailFlow data architecture as a coherent, modular and production-oriented platform.

The architecture connects:

```
customer events
→ event streaming
→ validation
→ PostgreSQL storage
→ governance
→ analytics
→ machine learning
→ API serving
→ Streamlit dashboards
→ observability
```

The platform demonstrates the main capabilities expected from a modern data architecture:

- structured data modeling;
- real-time ingestion;
- governed data storage;
- AI integration;
- orchestration;
- monitoring;
- CI/CD;
- future-ready deployment planning.

The current architecture is intentionally designed for local reproducibility while remaining aligned with a future cloud and Kubernetes deployment path.

This makes RetailFlow both demonstrable and extensible.

